CS612 · ADVANCED ALGORITHMS · MSC · FALL 2026

Lecture 14
Approximation Algorithms, Algorithmic Trade-offs & Course Synthesis

When exact is out of reach: guarantees you can prove, heuristics you can measure, and how to decide.

Saturday 26 December 2026 · 3 hours

PS4 due today · Presentations + vivas in separate slots (timetable announced)

Motivating problem: the deadline is Monday

A telecom operator must place monitoring probes on routers so that **every link is watched by a probe at one of its ends**. That is vertex cover on 50,000 routers and 300,000 links.

- L13: the problem is NP-hard. The exact search trees are exponential in the cover size (thousands).
- Management: "Give us a plan by Monday, and tell us how far from the best possible it could be."

> **DISCUSS · 3 MIN**
>
> What kind of answer can you honestly give? Is "our heuristic usually works well" acceptable? What would a **guarantee** look like?

BLOCK A · 00:10–01:00

Approximation guarantees

Ratios · vertex cover · set cover · limits

Approximation ratio; guarantee vs. heuristic

DEFINITION

A polynomial-time algorithm A is a ρ**-approximation** ($\rho \geq 1$) if for **every** instance I:

- minimisation: $A(I) \leq \rho \cdot \mathrm{OPT}(I)$;
- maximisation: $A(I) \geq \mathrm{OPT}(I)/\rho$.

The proof pattern: find a quantity $L(I)$ that the algorithm can compute with $L(I) \leq \mathrm{OPT}(I)$ (a **lower bound**), and show $A(I) \leq \rho\, L(I)$.

	Approximation algorithm	Heuristic
Quality	proved for every input	measured on some inputs
Typical result	often much better than ρ	often excellent
Adversarial inputs	still within ρ	can be arbitrarily bad
Evidence	a theorem	experiments (and their threats to validity)

Vertex cover: two 2-approximations

1 · Maximal matching

Repeatedly take an uncovered edge and add **both** endpoints.

Lower bound: the chosen edges form a matching M; any cover needs a distinct endpoint of each, so $\mathrm{OPT} \geq |M|$. The output has $2|M| \leq 2\,\mathrm{OPT}$ vertices.

Tight: a perfect matching graph gives $2n$ vs. n.

2 · LP rounding (weighted)

Solve the LP relaxation: minimise $\sum w_v x_v$ subject to $x_u + x_v \geq 1$ for every edge and $0 \leq x \leq 1$. Output $C = \{v : x_v \geq \tfrac{1}{2}\}$.

- A cover: each edge has an endpoint with $x \geq \tfrac{1}{2}$.
- $w(C) \leq \sum_{v \in C} w_v \cdot 2x_v \leq 2\,\mathrm{LP} \leq 2\,\mathrm{OPT}$ (the LP is a relaxation, so $\mathrm{LP} \leq \mathrm{OPT}$).

> Better than 2? Not known. It is NP-hard to do better than $\sqrt{2} \approx 1.414$ (Khot–Minzer–Safra 2017), and under the Unique Games Conjecture, factor $2 - \varepsilon$ is impossible.

Greedy set cover: an H_n-approximation

Universe U ($|U| = n$), sets $S_1..S_t$. Greedy: repeatedly pick the set covering the **most uncovered** elements.

THEOREM

Greedy uses $\leq H_n \cdot \mathrm{OPT}$ sets, where $H_n = \sum_{i=1}^{n} \frac{1}{i} \leq \ln n + 1$.

PROOF (PRICING)

When greedy picks a set covering c new elements, charge each of them $\frac{1}{c}$. The total charge = the number of greedy sets. Fix an optimal set S^* and list its elements in the order greedy covers them: $e_1, \ldots, e_d$. When e_j is covered, at least $d - j + 1$ elements of S^* are still uncovered, so greedy's choice covers $\geq d - j + 1$ new elements and e_j pays $\leq \frac{1}{d-j+1}$. So S^*'s elements pay $\leq H_d \leq H_n$. Summing over the OPT sets, which cover everything, gives greedy $\leq H_n \cdot \mathrm{OPT}$.

■

CAN WE DO BETTER?

No, essentially: approximating set cover within $(1 - \varepsilon) \ln n$ is NP-hard (Feige 1998; Dinur–Steurer 2014). Greedy is essentially optimal among polynomial algorithms.

TIGHTNESS

There are instances where greedy is $\Theta(\log n)$ times optimal: two disjoint "row" sets cover everything, while greedy picks a sequence of "column" sets of sizes $2^{k-1}, \ldots, 2, 1$.

Exercise A: guarantees

1. Show that the maximal-matching algorithm is **not** a 2-approximation for *weighted* vertex cover. Why does LP rounding still work?
2. **Max-coverage:** choose k sets to cover as many elements as possible. Greedy (k steps) achieves $\geq (1 - 1/e)\,\mathrm{OPT}$. Explain in words why each greedy step closes at least a $1/k$ fraction of the remaining gap to OPT, and why that gives $1 - (1 - 1/k)^k \geq 1 - 1/e$.
3. A heuristic returned covers of size 102 on 50 benchmark graphs whose optimum is known to be 100. A colleague claims it is a "1.02-approximation". What is wrong with the claim?

Pairs · 8 min

BLOCK B · 01:10–02:00

TSP, schemes, and heuristics

Metric TSP · impossibility · FPTAS · local search

Metric TSP: the double-tree 2-approximation

Metric: $d(u, w) \le d(u, v) + d(v, w)$ (the triangle inequality).

1. Compute an MST T (L09).
2. Walk around T (a DFS preorder), visiting each vertex the first time it is seen, i.e. **shortcut** repeated vertices.

PROOF

- **Lower bound:** removing one edge from an optimal tour gives a spanning tree, so $w(T) \le \mathrm{OPT}$.
- The full walk traverses each tree edge twice: cost $2w(T)$.
- Shortcutting never increases the cost (triangle inequality).
- So the tour costs $\le 2w(T) \le 2\,\mathrm{OPT}$.

■

CHRISTOFIDES (1976)

MST + a minimum-weight perfect matching on the odd-degree vertices + an Euler tour + shortcuts: a **1.5**-approximation. (Recently improved to $1.5 - 10^{-36}$: Karlin, Klein, Oveis Gharan 2021.)

NO TRIANGLE INEQUALITY: NO APPROXIMATION

For every constant ρ, a ρ-approximation for general TSP would decide Hamiltonian cycle (L13 exercise B2: weight 1 on edges, $\rho n + 1$ on non-edges). So none exists unless P = NP.

Approximation schemes: the knapsack FPTAS

Recall L07: the value-indexed DP runs in $O(n \cdot \sum v_i)$. Values are the problem, so scale them down.

1. Let $v_{\max}$ be the largest value (items heavier than W removed), and $K = \varepsilon v_{\max}/n$.
2. Rounded values $v'_i = \lfloor v_i / K \rfloor$ (at most n/ε).
3. Solve exactly with the value DP on v': $O(n \cdot n \cdot n/\varepsilon) = O(n^3/\varepsilon)$.

GUARANTEE

Rounding loses $< K$ per item, so $< nK = \varepsilon v_{\max} \le \varepsilon\,\mathrm{OPT}$ in total. The solution found has value $\ge (1-\varepsilon)\mathrm{OPT}$. ∎

WHY THIS IS POSSIBLE FOR KNAPSACK BUT NOT FOR TSP

Knapsack is only **weakly** NP-hard: its hardness lives in the large numbers, and rounding removes them at a controlled cost.

Strongly NP-hard problems have no FPTAS (unless P = NP). The L07/L13 distinction pays off here.

Heuristics and local search

- **Constructive heuristics:** nearest neighbour (TSP), greedy by density (knapsack), and so on. Fast, and no (constant) guarantee.
- **Local search:** start from a solution; apply improving moves until none exists (a *local optimum*). TSP 2-opt: replace edges $(a, b), (c, d)$ by $(a, c), (b, d)$ if that is shorter.
- **Metaheuristics:** simulated annealing, tabu search, genetic algorithms. Escape local optima; tuned empirically.
- **Randomisation:** random restarts, randomized rounding of LPs (which sometimes *does* give guarantees).

WHAT LOCAL SEARCH GUARANTEES

Only local optimality with respect to the moves. 2-opt on metric TSP has worst-case ratio $\Theta(\log n / \log \log n)$, and it can take exponentially many steps in the worst case.

Yet on real and random instances 2-opt is often within a few percent of optimal. That is an empirical fact to be **measured**, with the input family stated.

Experiment: guarantee vs. observed quality (Euclidean TSP)

$n = 12$ random points, 100 instances: ratio to the exact optimum (Held–Karp)			
Algorithm	**mean**	**worst**	**guarantee**
Double tree (MST)	1.205	1.418	2
Nearest neighbour	1.117	1.537	none (constant)
NN + 2-opt	**1.009**	**1.105**	**heuristic**

$n = 1000$: tour length / MST weight (an upper bound on the ratio to OPT)	
Double tree	1.566
Nearest neighbour	1.393
NN + 2-opt	**1.196**

`Student_Resources/tsp_approx_bench.cpp`, g++ 15.2 -O2.

INTERPRET · 5 MIN

1. The algorithm *with* a guarantee is the worst in practice. Contradiction?

2. Why does the table for $n = 1000$ use the MST, and what does that make the numbers?

3. What would you ship, and what would you report alongside it?

Exercise B: design with guarantees

1. Show that the double-tree algorithm's factor 2 is essentially tight: describe a metric instance where the shortcut tour costs close to $2\,\mathrm{OPT}$.
2. **k-center:** choose k facilities among n points (metric) to minimise the maximum distance of any point to its nearest facility. Farthest-first traversal (repeatedly add the point farthest from the chosen ones) is a 2-approximation. Sketch why. (Hint: the $k+1$ points chosen so far are pairwise far apart.)
3. For the knapsack FPTAS with $n = 1000$ and $\varepsilon = 0.01$: estimate the DP table size. Is it practical? What would you do instead?

Groups · 8 min

BLOCK C · 02:10–02:55

Trade-offs and course synthesis

Responses to hardness · a decision procedure for new problems

Responding to an NP-hard problem: the menu

Response	Guarantee	Cost	When it fits	CS612
Exact exponential (DP over subsets, search trees)	optimal	c^n or c^k	small n, or a small parameter k	L13
Branch-and-bound / ILP solvers	optimal (if it finishes)	unpredictable; exponential in the worst case	structured instances, good bounds	L13
Pseudo-polynomial DP	optimal	poly(n, numbers)	weakly NP-hard, small numbers	L07
Special structure	optimal	polynomial	trees, bipartite, planar, intervals, …	L08–L12
Approximation algorithms / schemes	within ρ (or $1 \pm \varepsilon$)	polynomial	a provable bound is needed	L14
Heuristics / local search / metaheuristics	none (measured)	tunable	quality matters, guarantees don't	L14
Randomised algorithms	expected / w.h.p.	polynomial	sampling, rounding, restarts	L03

Course synthesis: facing a new computational problem

1. **Model:** input, output, input size (bits!), cost model. What is fixed, and what varies (L01, L07)?
2. **Assumptions:** weights, adversary, distributions, static vs. dynamic, sparse vs. dense, metric, small parameters (L04, L10).
3. **Paradigm:** D&C (L02), randomization (L03), augmentation / amortization (L05), DP state design (L06–L07), greedy/matroid (L08), graphs and flow (L09–L12), strings (L11).
4. **Correctness:** invariants, induction, exchange, cut/cycle, reductions (L01, L08, L13).
5. **Complexity:** worst/expected/amortized/w.h.p./approximate, and the lower bound (L01–L05, L09).
6. **Hardness:** is an exact efficient algorithm plausible? Weak or strong (L07, L13)?
7. **Alternatives:** when does each one win?
8. **Evidence:** hypotheses first; controlled inputs that stress the assumptions; spread; threats to validity (the project).

THE RECURRING LESSON

A result is a **claim + its guarantee type + its assumptions + its evidence**.

"Dijkstra is $O(m \log n)$" is incomplete. "Dijkstra with a binary heap is $O((n + m) \log n)$ worst case for non-negative weights, and on sparse random graphs it beat Bellman–Ford only by a constant, because of early termination" is an MSc answer.

Exercise C: the whole procedure on one problem

Problem. A city places electric-vehicle chargers at intersections. Every road segment must have a charger within distance D of both its endpoints, installation costs differ by site, and the budget is fixed. Planners want the cheapest plan, and they want to know how good it is.

1. Model it. Which classic problem(s) does it resemble? (Think about L13–L14.)
2. What is the input size? Which assumptions matter (metric distances? planar road network?)
3. Is an exact efficient algorithm plausible? Argue by a reduction sketch.
4. Propose two responses from the menu, with their guarantees.
5. Design one experiment to compare them, with a hypothesis and what would refute it.

Groups · 15 min, then 2–3 groups present (5 min)

Close: the final exam and beyond

Final exam (60 marks, written, no MCQ)

- A: short analytical reasoning (10) · B: design + correctness proof (14) · C: analysis (10) · D: a graph/string scenario (12) · E: NP-completeness, reductions, approximation (14)
- Pseudocode, not code. Every bound with its type. Every reduction with its direction and both implications.
- Prepare with the in-class exercises, PS1–PS4, and one page per lecture: the algorithm, its guarantee, its assumption, and the alternative.

WHERE THIS LEADS

- Randomized and online algorithms, streaming and sketching
- Parameterised and fine-grained complexity
- LP/SDP-based approximation, hardness of approximation
- Algorithm engineering: making theory fast in practice

Thank you. Good luck with the vivas and the final.